□ 1-HOUR PLAN (Follow this exactly)

□ 0–15 min → Core Concepts (VERY IMPORTANT)

□ 15–45 min → Code Patterns for each Agent

□ 45–60 min → Practice + exam strategy

🔥 PART 1: CORE IDEA (SUPER SIMPLE)

Everything in your lab is based on this structure:

```
percept = environment.get_percept()
action = agent.act(percept)
```

👉 That's it. Every question is just a variation of this.

□ 1. WHAT IS AN AGENT?

From your lab:

👉 Agent = decision maker
👉 Environment = world
👉 Sensors = input
👉 Actuators = output

□ TYPES OF AGENTS (THIS IS THE KEY FOR EXAM)

1□ $imple Reflex Agent

👉 Uses ONLY current input
👉 NO memory

💡 Example: Light ON/OFF (Task 1)

```
def act(self, percept):
    if percept == "Occupied":
        return "Turn ON Light"
```

```
    else:
        return "Turn OFF Light"
```

✓ No history
✓ Instant reaction

---

2 Model-Based Agent

☞ Uses MEMORY

💡 From your Task 2:
Robot remembers which shelves were empty

```
self.memory = []

def act(self, percept):
    if percept == "Empty":
        self.memory.append("checked")
```

✓ Stores past
✓ Avoids repetition

---

3 Goal-Based Agent

☞ Works toward a goal

💡 Example: deliver medicine

```
goal = "Deliver Medicine"

if not delivered:
    move_to_room()
```

✓ Focus = achieving goal

---

4 Utility-Based Agent

☞ Chooses BEST option

💡 Task 3 (Drone)

```
utility = speed - risk - battery_usage
```

✓□ Compares options
✓□ Picks highest utility

□ PART 2: HOW TO IDENTIFY TASK TYPE (VERY IMPORTANT)

From your tasks file:

Task	Agent Type	WHY
Task 1 (Lights)	Reflex	No memory
Task 2 (Robot shelves)	Model-based	Uses memory
Task 3 (Drone)	Utility	Trade-offs
Security tasks	Mix (mostly reflex + utility)	
Hospital robot	Goal-based	Has goal

💻 PART 3: CODE TEMPLATE (USE THIS IN EXAM)

🔄 UNIVERSAL TEMPLATE

```
class Environment:
    def __init__(self):
        pass

    def get_percept(self):
        return something


class Agent:
    def __init__(self):
        pass

    def act(self, percept):
        return action


def run():
    env = Environment()
    agent = Agent()

    for i in range(5):
        percept = env.get_percept()
        action = agent.act(percept)
        print(percept, action)
```

👉 You just MODIFY logic per question.

⚡ PART 4: PRACTICE QUICK (IMPORTANT)

✅ TASK 1 (Lights → Reflex Agent)

💡 Logic:

- If motion → ON
- Else → OFF

```
class LightAgent:
    def act(self, percept):
        if percept == "Motion":
            return "ON"
        else:
            return "OFF"
```

✅ TASK 2 (Warehouse → Model-Based)

💡 Key point:
👉 MUST store checked shelves

```
class RobotAgent:
    def __init__(self):
        self.checked = []

    def act(self, percept):
        if percept == "Item Found":
            return "Pick Item"
        else:
            self.checked.append("Empty Shelf")
            return "Move Next"
```

✅ TASK 3 (Drone → Utility-Based)

💡 Core idea:
👉 Compare actions

```
def utility(speed, battery, risk):
    return speed - battery - risk
if fast > slow and fast > route:
    return "Fly Fast"
```

🎯 PART 5: EXAM CHEAT STRATEGY

🔑 ALWAYS DO THIS:

1. Identify agent type
2. Write Environment
3. Write Agent class
4. Write act() logic
5. Run loop

□ MEMORY TRICK

- Reflex → “No brain”
- Model → “Remembers”
- Goal → “Has target”
- Utility → “Chooses best”

🚀 LAST 10-MIN REVISION

If nothing else, remember:

```
if percept == condition:
    action = something
```

☞ 80% of your lab = this pattern

✅ TASK 1 — Light Control System (Simple Reflex Agent)

💡 Type: **Reflex Agent** (no memory)

```
import random
import time

class Environment:
    def __init__(self):
        self.rooms = ["Room1", "Room2", "Room3"]

    def get_percept(self):
        # Randomly simulate motion in rooms
        return {room: random.choice(["Occupied", "Empty"]) for room in self.rooms}


class LightAgent:
    def act(self, percept):
```

```
        actions = {}
        for room, state in percept.items():
            if state == "Occupied":
                actions[room] = "Light ON"
            else:
                actions[room] = "Light OFF"
        return actions


def run():
    env = Environment()
    agent = LightAgent()

    for i in range(5):  # simulate continuous system
        percept = env.get_percept()
        action = agent.act(percept)

        print(f"\nStep {i+1}")
        print("Percept:", percept)
        print("Action:", action)
        time.sleep(1)


run()
```

✅ TASK 2 — Warehouse Robot (Model-Based Agent)

💡 Type: **Model-Based Agent** (has memory)

```
import random

class Environment:
    def __init__(self):
        self.shelves = ["A", "B", "C", "D", "E"]
        self.items = {shelf: random.choice([True, False]) for shelf in self.shelves}

    def get_percept(self, shelf):
        return self.items[shelf]


class RobotAgent:
    def __init__(self):
        self.checked_shelves = []
        self.collected_items = []

    def act(self, shelf, percept):
        if shelf in self.checked_shelves:
            return "Skip (Already Checked)"

        self.checked_shelves.append(shelf)

        if percept:
            self.collected_items.append(shelf)
            return f"Pick item from shelf {shelf}"
        else:
```

```
        return f"No item at shelf {shelf}"


def run():
    env = Environment()
    agent = RobotAgent()

    print("Initial Shelf State:", env.items)

    for shelf in env.shelves:
        percept = env.get_percept(shelf)
        action = agent.act(shelf, percept)

        print(f"Shelf {shelf} → Percept: {percept}, Action: {action}")

    print("\nCollected Items:", agent.collected_items)


run()
```

✅ TASK 3 — Drone Delivery (Utility-Based Agent)

💡 Type: **Utility-Based Agent**

```
import random

class Environment:
    def get_percept(self):
        return {
            "battery": random.randint(20, 100),
            "weather": random.choice(["Clear", "Windy", "Rainy", "Stormy"]),
            "distance": random.randint(1, 20),
            "obstacles": random.randint(0, 5)
        }


class DroneAgent:
    def utility(self, action, percept):
        battery = percept["battery"]
        weather = percept["weather"]
        distance = percept["distance"]
        obstacles = percept["obstacles"]

        if action == "Fly Faster":
            return 10 - (100 - battery)/10 - obstacles*2 - (5 if weather != "Clear" else 0)

        elif action == "Fly Slower":
            return 6 + battery/20 - obstacles

        elif action == "Alternative Route":
            return 8 - distance/2 + (3 if obstacles > 2 else 0)

    def act(self, percept):
        actions = ["Fly Faster", "Fly Slower", "Alternative Route"]
```

```
        utilities = {action: self.utility(action, percept) for action in actions}

        best_action = max(utilities, key=utilities.get)

        return best_action, utilities


def run():
    env = Environment()
    agent = DroneAgent()

    for i in range(5):
        percept = env.get_percept()
        action, utilities = agent.act(percept)

        print(f"\nStep {i+1}")
        print("Percept:", percept)
        print("Utilities:", utilities)
        print("Chosen Action:", action)


run()
```